

TimeLoom – Productivity Visualizer

A Python-Based Console Application for Time & Productivity Tracking

G.Vinod Kumar (24X01A6252)
Cyber Security - A



Introduction

TimeLoom is a lightweight Python console application that helps users track, manage, and analyze how they spend time across study, work, and breaks. Implemented using core Python concepts for clarity, portability, and ease of extension.

Problem Statement — Why TimeLoom?

- Many users lack simple tools to record and analyze daily time allocation.
- Opportunity to apply essential Python skills in a practical project.
- Provides a real-world, extensible solution that can scale to GUI or web later.
- Builds coding confidence through end-to-end design: data, logic, and reporting.





Objectives

- Implement a menu-driven time tracking system (console).
- Support category-based organization (study, work, breaks, custom).
- Persist data using JSON for portability and simplicity.
- Produce analytics and human-readable reports (including ASCII heatmap).
- Keep the tool lightweight, user-friendly, and easy to extend.

System Architecture — Block Diagram



The diagram shows a linear flow from user input through modular controllers to JSON storage and final summary outputs (reports, weekly heatmap).



Modular Design Breakdown



Main module

main.py — Entry point and menu controller; routes user actions to modules and handles session flow.



Storage

storage.py — JSON read/write helpers, schema validation, and backup routines.



Tracker

tracker.py — Timer functions, entry logging, category management, edit/delete, and search.



Analytics

analytics.py — Summaries, reports, aggregation, and ASCII weekly heatmap generation.



I/O Operations

io_ops.py — Import/export CSV and JSON, simple transformations, and validation.

Key Features

- Start/Stop interactive timer and manual time logging.
- Edit, delete, and search time entries by category or date.
- Category management with custom labels and colors.
- Persistent JSON storage (data.json) with optional backups.
- Generate analytics: summaries, top activities, and weekly ASCII heatmap.
- CSV/JSON import-export for external analysis.





How It Works — User Flow

1. Run **main.py** to start the console application.
2. Menu appears with 13 options (track, log, edit, reports, import/export, settings).
3. User selects an action (e.g., start timer, log manual entry, view report).
4. Operations call storage helpers — changes saved automatically to data.json.
5. User generates reports or exports data as needed.



Console-based menu and analytics report

Sample Output (Placeholder)

Example elements shown in the screenshot:

- Menu: Start Timer | Stop Timer | Log Entry | View Reports | Export
- Report: Daily summary table and top 3 categories by time
- ASCII Weekly Heatmap: rows=days, columns=hours — intensity as characters

Conclusion & Future Scope

TimeLoom demonstrates practical application of Python fundamentals (lists, dicts, loops, file I/O, modular design) to solve a real productivity problem in a compact, extensible package.

Future enhancements

- GUI with Tkinter or cross-platform framework for visual interaction.
- Web app using Flask/Django with multi-user authentication and cloud storage.
- Rich visual charts via matplotlib or Plotly and exportable PDF reports.
- Advanced analytics: pattern detection, suggestions, and goal tracking.

Recommended: include a short README and sample data.json for evaluation and reproducibility.

